

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS:

Total Marks: 20

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type:
text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld
    </div>
  </body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

*I Y. Abhay-192411219(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Y. Abhay

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

CSA4305-Internet Programming

Name: Y. Abhay | Reg No: 192411219

Case Study 1: HTTP Response Analysis

Given HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

1. Incorrect or invalid fields in the header

- Content-Length: -150 is invalid — the Content-Length header must be a non-negative integer representing the number of bytes in the response body; a negative value has no meaning.
- Apart from that, the other fields are syntactically valid: the status line, Date, Content-Type, and Connection headers all follow correct HTTP/1.1 formatting.
- No other headers such as Server, Cache-Control, or Content-Encoding are present, but their absence is not an error — they are optional.

2. Issue with Content-Length and its impact

Content-Length must state the exact size, in bytes, of the message body so the client (browser) knows exactly when the response ends — this matters most on persistent (keep-alive) connections, where the underlying TCP connection is reused for multiple requests and there is no other signal marking the end of one response and the start of the next.

- A negative value is not parseable as a byte count, so a strict HTTP client/parser will treat the header as malformed and may reject the response outright.
- If a lenient client ignores the invalid value, it no longer knows where the response body ends. On a keep-alive connection this leads to response smuggling / desynchronization: the client may read into the next response's bytes, corrupting both responses, or hang waiting for data that never comes.
- This directly impacts rendering — the browser may show a truncated or garbled page, or the connection may stall until it times out.

3. Does the status code match the response correctness?

No. The 200 OK status line asserts that the request succeeded and a valid representation of the resource is being returned, but a response carrying an invalid Content-Length is not a well-formed HTTP message. The status code and the malformed header are contradictory: the server is claiming success while sending a header that a compliant client cannot reliably interpret. In practice, this response should not have been sent as 200 OK without first fixing the Content-Length.

4. Suggested corrections

- Set Content-Length to the actual byte length of the response body, e.g. Content-Length: 150 (a non-negative integer).
- Alternatively, if the body length is not known in advance (e.g., streamed content), omit Content-Length and use Transfer-Encoding: chunked instead.

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
```

Content-Length: 150
Connection:keep-alive

5. Validate header values server-side before dispatching the response, so a negative or non-numeric Content-Length can never be emitted.

Case Study 2: DOM Structure Analysis

Given DOM tree / HTML:

```
<html>
  <head>
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld
    </div>
  </body>
</html>
```

1. Missing or improperly closed elements

- No `<!DOCTYPE html>` declaration at the top of the document.
- `<html>` has no `lang` attribute, and `<head>` has no `<meta charset>`.
- `<p>HelloWorld` is never closed with `</p>`.
- The `<div>` tag is closed immediately after the unclosed `<p>`, so the parser must decide whether `</div>` also implicitly closes the still-open `<p>`.

2. How the browser will interpret this structure

Per the HTML5 parsing specification, browsers use well-defined error-recovery ('tag soup') rules rather than rejecting the page. When the parser encounters `</div>` while a `<p>` element is still open, it implicitly closes the open `<p>` first (because a `<p>` cannot legally straddle the close of its parent), then closes the `<div>`. So the browser's generated DOM effectively becomes:

```
<div>
  <p>HelloWorld</p>
</div>
```

- The visible result looks correct because auto-closing happens to produce the intended nesting here, but this is the parser silently repairing an authoring mistake, not a guarantee — different or more complex malformed nesting can produce inconsistent DOMs across parser implementations.

3. Impact on rendering and layout

- For this specific simple case, the visual output is largely unaffected, since auto-closing restores the intended structure.
- However, relying on this behaviour is risky: CSS selectors and JavaScript DOM queries (e.g., `div > p`, or `childNodes` traversal) assume a specific tree shape, and if the auto-closing logic differs from what the author intended, layout, spacing, and script logic can silently break.
- Without a `DOCTYPE`, the page may render in Quirks Mode, altering box model calculations (margins, padding, widths) compared to Standards Mode, which can shift the layout unexpectedly.

- Malformed/unclosed tags also make the page harder to maintain and more fragile — a small future edit can expose incorrect nesting that today's auto-repair happens to hide.

4. Corrections to make the DOM valid

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>TestPage</title>
</head>
<body>
  <div>
    <p>HelloWorld</p>
  </div>
</body>
</html>
```

Case Study 3: GET vs POST in Login System

Given comparison:

M e t h o d	U R L E x a m p l e	D a t a i n t y	S e c u r i t y L e v e l
GET	/login? username=ad min&p	Visible in URL	Low

	a		
	s		
	s		
	=		
	1		
	2		
	3		
	4		
P	/	H	M
O	l	i	o
S	o	d	d
T	g	e	e
	i	n	r
	n	i	a
		n	t
		b	e
		o	
		d	
		y	

1. Differences in data transmission between GET and POST

- GET appends all form fields as a query string on the URL (?user=admin&pass=1234), so the data becomes part of the resource address itself.
- POST places the form fields in the HTTP request body, separate from the URL, with the body's format described by the Content-Type header (e.g., application/x-www-form-urlencoded, multipart/form-data, or application/json).
- GET requests are idempotent and intended for retrieving data, are bookmarkable, and are cached by browsers/proxies; POST requests are intended for actions that change state and are not cached or bookmarked by default.
- GET has practical URL-length limits (browser/server dependent, often ~2000 characters); POST bodies can carry much larger payloads.

2. Which method is more suitable for login, and why

POST is the more suitable method for login. Credentials submitted via POST travel in the request body rather than the URL, so they are not written into browser history, bookmarks, server access logs, or the Referer header sent to other sites. POST also avoids the practical URL-length restrictions of GET and matches the semantic intent of HTTP — a login is an action with side effects (creating a session), which is what POST is designed for, whereas GET is meant for safe, repeatable retrieval of a resource.

3. Security risks of using GET for authentication

- Credential exposure in the URL — visible in the browser address bar and easily shoulder-surfed.
- Stored in browser history and in bookmarks if the user saves the page.
- Logged in plaintext by web servers, proxies, and load balancers, which routinely log full request URLs.
- Leaked via the Referer header to any third-party resource (scripts, images, analytics) loaded on the destination page.
- Cached by browsers and intermediate caching proxies, potentially exposing credentials to other users of a shared cache.
- More easily replayed by an attacker who has captured the URL, since the entire request can be re-issued from a saved link.

4. Best practices for secure login implementation

- Always use POST for submitting login credentials, never GET.
- Serve the login form and endpoint exclusively over HTTPS (TLS) so the POST body is encrypted end-to-end.
- Hash and salt passwords server-side with a slow algorithm (bcrypt, Argon2, or scrypt) — never store or log plaintext passwords.
- Use HttpOnly, Secure, and SameSite cookie attributes for session cookies to reduce XSS/CSRF exposure.
- Implement CSRF tokens, rate limiting/account lockout, and optionally multi-factor authentication.
- Avoid exposing detailed error messages (e.g., "invalid password" vs "invalid username") that let an attacker enumerate valid accounts.

Case Study 4: HTML Code Inspection

Given HTML:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>
```

1. Missing closing tags

- `<h1>Welcome` is not closed with `</h1>`.
- `<p>This is a paragraph` is not closed with `</p>`.
- `<html>` has no `lang` attribute (not a closing-tag issue, but a compliance gap worth noting).
- `<img>` is a void element, so it correctly has no closing tag — but see the self-closing/attribute issues noted below.

2. Issues with the `<img>` tag

- Missing the `alt` attribute — required for accessibility (screen readers) and displayed as fallback text if the image fails to load; its absence also fails WCAG compliance.
- Missing `width` and `height` attributes, which help the browser reserve layout space before the image loads and prevents content from shifting (layout shift) once it does.
- In strict XHTML/XML-served documents, the tag would additionally need to be self-closed as ``, though this is optional in standard HTML5.
- No indication of whether `image.jpg` exists relative to the current path, or a fallback if it fails to load.

3. How browsers will handle these errors

Browsers apply HTML5's error-recovery parsing rules rather than failing outright. Because `<img>` is a void element with no content, and `</body>` forces all open elements to close, the parser will implicitly close the still-open `<h1>` when it reaches the following `<p>`, and will implicitly close the `<p>` when it reaches `</body>` (or the next block-level element). The resulting DOM approximates:

```
<h1>Welcome</h1>
<p>This is a paragraph</p>

```

- The missing alt is not a parsing error at all — the page still renders — but it silently degrades accessibility and SEO, and if image.jpg fails to load, nothing meaningful is shown in its place.

4. Corrected version of the code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Sample</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph</p>
  
</body>
</html>
```

Case Study 5: Browser-Server Communication Flow

Given sequence: (1) User enters a URL → (2) DNS lookup → (3) HTTP request sent → (4) Server processes request → (5) Response returned → (6) Browser renders page.

1. Interpretation of each step

- Step 1 — User enters URL: the browser parses the URL into scheme, host, port, path, and query components.
- Step 2 — DNS lookup: the browser (via the OS resolver) translates the human-readable hostname into an IP address, checking local/browser cache, the OS cache, the router, and finally a recursive DNS resolver / authoritative name servers if needed.
- Step 3 — HTTP request sent: the browser opens a TCP connection to that IP (performing a TLS handshake first if HTTPS), then sends an HTTP request line, headers, and optional body.
- Step 4 — Server processes request: the web/application server routes the request, runs any business logic, queries a database if needed, and builds the response.
- Step 5 — Response returned: the server sends back a status line, headers, and body over the same connection.
- Step 6 — Browser renders page: the browser parses the HTML into the DOM, fetches linked sub-resources (CSS, JS, images), builds the render tree, and paints the final page — executing any JavaScript along the way.

2. Where delays can occur

- DNS resolution delay — an uncached lookup traversing multiple resolver hops.
- TCP/TLS handshake latency — extra round trips to establish and secure the connection, especially on high-latency or first-time connections.
- Server processing time — slow database queries, unoptimized backend code, or server overload.
- Network transmission time — bandwidth limits and physical distance between client and server.
- Rendering/parsing delay — large HTML/CSS/JS payloads, render-blocking scripts and stylesheets, and additional round trips to fetch sub-resources (images, fonts, third-party scripts).

3. Role of DNS in the process

DNS acts as the internet's directory service — it converts the human-friendly domain name typed by the user into the numeric IP address the network actually needs to route packets to. Without a successful DNS lookup, the browser has no address to send the HTTP request to, so this step is a hard prerequisite that gates every subsequent step; any delay or failure here (e.g., a slow or unresponsive resolver) delays or blocks the entire page load.

4. Optimizations to improve performance

- DNS prefetching (`<link rel="dns-prefetch">`) and caching to avoid repeated lookups.
- Reuse connections via HTTP keep-alive, and use HTTP/2 or HTTP/3 to multiplex requests over fewer connections and reduce handshake overhead.
- Use a Content Delivery Network (CDN) to serve static assets from a server geographically closer to the user.
- Minify and compress HTML/CSS/JS (e.g., gzip/Brotli) to shrink transfer size.
- Load non-critical JavaScript with `async/defer` so it doesn't block HTML parsing and rendering.
- Apply server-side caching and database query optimization to reduce backend processing time.
- Optimize and lazy-load images, and set explicit width/height to avoid layout shift during render.